

**From Adversarial Attacks to Robust Classifiers - A Study in Social
Media Spam Detection - Black Box & White Box**

Jonathan Jose Penaloza Rumie

Advisor- Dr. Sayani Sarkar

Reader- Dr. Andrew Karem

Computer Science Department

April 05, 2025

Table of Contents

- 1. Introduction**
- 2. Background**
- 3. Methodology**
 - 3.1 Dataset Creation**
 - 3.2 White and Black Attacks on the Dataset**
 - 3.3 Exploratory Data Analysis (EDA)**
 - 3.4 Machine Learning Models**
- 4. Experiments and Results**
 - 4.1 Hyperparameter Configuration**
 - 4.2 Evaluation Metrics**
 - 4.3 Results**
- 5. Conclusion**
- 6. Future Scope**
- 7. References**

List of Figures

- Figure 1: Label Distribution of the Original Spam Dataset
- Figure 2: Label Distribution of the Attacked Dataset
- Figure 3: General Word Cloud of the Black Box Attacked Dataset
- Figure 4: Word Cloud of Ham Messages (Black Box)
- Figure 5: Word Cloud of Spam Messages (Black Box)
- Figure 6: Word Cloud of Spam Messages (White Box)
- Figure 7: Polarity Distribution of Black Box Adversarial Attack
- Figure 8: Polarity Distribution of White Box Adversarial Attack
- Figure 9: Histogram of Average Word Length (Black Box)
- Figure 10: Histogram of Average Word Length (White Box)
- Figure 11: Histogram of Text Length (Black Box)
- Figure 12: Histogram of Text Length (White Box)
- Figure 13: Stop Word Frequency of Black Box and White Box Attacks
- Figure 14: N-Gram Analysis of Black Box Attack
- Figure 15: N-Gram Analysis of White Box Attack
- Figure 16: TF-IDF Score Distribution of Black Box Attack
- Figure 17: TF-IDF Score Distribution of White Box Attack

List of Tables

- Table I: Dataset creation steps with Black Box Adversarial Attack
- Table II: Dataset creation steps with White Box Adversarial Attack
- Table III: Hyperparameters for machine learning models used in Black Box Adversarial Attack
- Table IV: Hyperparameters for machine learning models used in White Box Adversarial Attack
- Table V: Performance Metrics for Ham and Black Box Spam for different machine learning models
- Table VI: Performance Metrics for Ham and White Box Spam for different machine learning models

1. Introduction

In the rapidly evolving landscape of natural language processing (NLP), sentiment analysis has emerged as a critical tool for understanding and interpreting human emotions expressed in text. This capability has become increasingly vital in our digital age, where vast amounts of textual data are generated daily through social media, customer reviews, and online interactions. Sentiment analysis not only provides valuable insights for businesses and organizations but also plays a crucial role in enhancing user experiences and informing decision-making processes across various domains.

However, as the importance and ubiquity of sentiment analysis grows, so too does the threat of adversarial attacks on the machine learning models that power these systems. These attacks, which can be categorized into white-box and black-box varieties, pose a significant challenge to the reliability and security of sentiment classification models. White-box attacks, leveraging full knowledge of the model's architecture and parameters, exploit gradient information to craft subtle perturbations that can dramatically alter a model's predictions. In contrast, black-box attacks operate with limited access to the model, relying on heuristic methods and model queries to generate adversarial examples that can deceive classifiers.

The implications of these adversarial attacks extend far beyond academic interest, intersecting critically with the fields of data science and cybersecurity. As organizations increasingly rely on sentiment analysis for customer insights, brand management, and even financial decision-making, the vulnerability of these systems to malicious manipulation becomes a pressing concern. Adversarial attacks on sentiment classifiers could lead to misinformed business strategies, compromised user trust, and potentially significant financial losses.

This thesis aims to address a fundamental question at the intersection of NLP, machine learning, and cybersecurity: how do black-box and white-box attacks impact sentiment classification models? By exploring this question, we seek to not only advance our understanding of the vulnerabilities inherent in current sentiment analysis systems but also to contribute to the development of more robust and resilient models.

Our research leverages a novel dataset of social media messages, carefully crafted to include both original and adversarially attacked versions. This dataset created using a range of techniques including synonym replacement, character-level perturbations, and spammy phrase insertion, provides a realistic testbed for evaluating the effectiveness of various adversarial attack strategies. Furthermore, by employing traditional machine learning models such as Logistic Regression, Decision Trees, Random Forests, K-Nearest Neighbors, Bagging, Gradient Boosting, and Support Vector Machines, we establish a comprehensive baseline for assessing the impact of these attacks on different classification approaches.

The structure of this thesis will proceed as follows: First, we will provide a thorough background on sentiment analysis, adversarial machine learning, and the specific challenges posed by text-based attacks. Next, we will detail our methodology for creating the adversarial dataset and implementing both white-box and black-box attacks. We will then present our experimental results, analyzing the effectiveness of different attack strategies across various sentiment classification models. Finally, we will discuss the implications of our findings for the fields of NLP and cybersecurity, offering insights into potential defense mechanisms and directions for future research.

By bridging the gap between sentiment analysis and adversarial machine learning, this research not only contributes to academic discourse but also addresses critical real-world

challenges in maintaining the integrity and reliability of NLP systems. Our findings aim to inform people about the development of more robust sentiment classification models, ultimately enhancing the security and trustworthiness of text analysis systems in an increasingly complex digital landscape.

2. Background

Adversarial attacks exploit the inherent weaknesses in machine learning models by introducing subtle modifications to input data, thereby deceiving classifiers. In the context of image classification, these attacks manifest as small perturbations to the input images, which can significantly alter the model's predictions. The challenge of detecting such adversarial examples is heightened in black-box settings, where the attacker has limited knowledge of the model's internals. Guo et al. propose a simple yet effective method for constructing adversarial images in black-box scenarios. Their approach leverages continuous-valued confidence scores and employs a highly query-efficient algorithm that iteratively adjusts the input image. This method demonstrates unprecedented efficiency in both untargeted and targeted attacks, providing a strong baseline for future research in black-box adversarial attacks. Their work underscores the vulnerability of machine learning models to adversarial manipulation, even with minimal changes to the input features [1].

Similarly, Narodytska and Kasiviswanathan introduce a novel local-search technique to approximate the network gradient, which is then used to perturb a small set of pixels in the image. This technique demonstrates that deep neural networks remain vulnerable even when attackers have limited model visibility. The study strengthens the case for designing robust models and adds a complementary perspective to the work by Guo et al., reinforcing the broad susceptibility of visual classifiers in constrained attack settings [2].

Beyond the visual domain, adversarial vulnerabilities extend into natural language processing (NLP). Sentiment analysis, a key NLP application, identifies and extracts subjective information from text using natural language processing and text mining. It has become increasingly relevant with the rapid growth of Internet-based applications, such as social media platforms and blogs, resulting in an abundance of user-generated content. Wankhade et al. provide a comprehensive survey of sentiment analysis methods, applications, and challenges. Their work evaluates and compares a variety of approaches while highlighting the nuances of interpreting sentiments in informal, dynamic, and multilingual environments. The survey builds a foundation for future exploration, especially as sentiment models become targets of adversarial attacks [3].

Zhang et al. further delve into adversarial challenges in NLP by surveying methods that generate textual adversarial examples. The discrete nature of language data poses unique difficulties for both attackers and defenders. Their review details approaches for crafting text perturbations that are often imperceptible yet capable of flipping model outputs. The work emphasizes the critical need for defense mechanisms tailored to NLP, where interpretability and coherence are essential. It serves as a foundational resource for understanding adversarial landscapes within textual applications [4].

The increasing integration of large language models (LLMs) into cybersecurity has brought both promise and new concerns. Xu et al. present a systematic literature review on the use of LLMs in security contexts, analyzing over 30,000 papers and synthesizing insights from 127 high-impact studies. Their findings highlight LLM applications in vulnerability detection, malware analysis, phishing detection, and more. Notably, Section 8.1.3 of their study discusses challenges in LLM interpretability and ethical use. The authors stress that trust and

transparency are crucial, particularly when models are applied in high-stakes domains like cybersecurity. Their call for more interpretable and ethically grounded LLM applications signals a critical path forward in the secure use of AI [5].

In direct application to adversarial manipulation in NLP, Alzantot et al. propose a black-box, population-based optimization algorithm capable of generating semantically and syntactically similar adversarial examples. With success rates as high as 97% and 70% for sentiment analysis and textual entailment models respectively, their work highlights just how fragile NLP models can be. Their approach, relying on minimal but strategically placed changes, reaffirms the urgent need for more resilient NLP architectures [6].

While black-box threats dominate many discussions, white-box attacks—where the attacker has full knowledge of the model—present their own dangers. Ebrahimi et al. introduce HotFlip [7], a method based on atomic token swaps guided by gradients from the one-hot input vectors. Their method shows that just a few character- or word-level manipulations can drastically reduce a model’s accuracy. The work is notable not only for its technical precision but also for demonstrating how white-box access amplifies the potency of adversarial threats.

Zang et al. address adversarial NLP from a combinatorial optimization perspective. They develop a sememe-based word substitution model that ensures semantic and syntactic consistency while misleading the classifier. Using particle swarm optimization to efficiently search the space of substitutions, their method achieves high attack success rates across multiple tasks. This work exemplifies the growing sophistication of text-based attacks and calls attention to the nuanced challenge of defending language models while preserving linguistic coherence [8].

Further building on adversarial NLP, Vázquez-Hernández et al. propose an aspect-based adversarial attack strategy specifically for sentiment analysis. Their technique selectively modifies aspect-related expressions to deceive models while preserving the semantic essence of the text. The study demonstrates substantial drops in model accuracy with minimal textual perturbations, underlining the targeted and strategic evolution of modern adversarial methods [9].

Lastly, Goyal et al. take a broad view with a comprehensive survey of adversarial defenses in NLP. They introduce a novel taxonomy to classify defense mechanisms and examine attack types such as evasion, poisoning, and inference attacks. Their work outlines the weaknesses in current defense strategies and emphasizes the importance of robust, adaptable systems. Their contribution complements the adversarial attack research by highlighting the equally important and evolving landscape of adversarial defense [10].

3. Methodology

This document outlines a comprehensive and structured methodology for evaluating the robustness of machine learning models against adversarial attacks in the context of spam detection. The focus is placed on comparing two distinct types of adversarial strategies—black-box and white-box attacks—and measuring how well various classifiers perform under these challenging scenarios. This section details the processes of dataset creation, adversarial attack implementation, model training, and performance evaluation. The methodology aims to simulate realistic adversarial conditions and provide insights into which models are best suited to withstand manipulation while maintaining high predictive accuracy and generalization.

This research adopts a rigorous experimental framework to investigate how adversarial attacks affect the performance of machine learning models, particularly in sentiment and spam classification tasks. The methodology is carefully designed to reflect practical challenges encountered in real-world spam detection systems, including the increasing sophistication of adversarial techniques used by malicious actors to bypass content filters. By incorporating a variety of attack types and maintaining controlled experimental conditions, the study ensures a fair and meaningful comparison of different classifiers under both normal and adversarial settings. This approach allows for a thorough exploration of each model's vulnerabilities and strengths, thereby informing more robust spam detection strategies.

3.1. Dataset Creation

The foundation of this study is built upon a dataset derived from the Twitter Spam Detection project, which initially includes 5,169 unique text samples. These messages are categorized into two classes: spam and ham (legitimate). In this context, "ham" refers to legitimate messages that are usually found in nature, non-spam messages, as opposed to spam. Of the entire dataset, 13% of the messages are labeled as spam, while the remaining 87% are classified as ham. Although the dataset is well-structured and labeled, its original form does not sufficiently capture the nuances required to evaluate the effectiveness of adversarial attacks. Therefore, to enhance its relevance for this study, a novel and more challenging dataset was constructed by introducing a variety of adversarial perturbations to legitimate (ham) messages.

To overcome this limitation, we augmented the dataset by incorporating adversarial perturbations, particularly targeting legitimate (ham) messages. This modification was essential for simulating real-world conditions in which malicious entities attempt to deceive spam filters by subtly altering benign messages. Perturbation techniques employed include

synonym replacement (e.g., changing “win” to “earn”), character-level modifications (such as random swaps or deletions), and the insertion of misleading spam phrases (e.g., “claim prize,” “big win,” “unlock reward”).

The design rationale behind attacking ham messages lies in the typical modus operandi of adversaries in the wild—disguising spam messages under the facade of innocuous language. By transforming legitimate messages into adversarial examples, we ensured the models would be tested not just on their ability to identify traditional spam, but also on their aptitude for resisting manipulated inputs that closely resemble ham. The resulting adversarial dataset consists of original (unaltered) and modified (attacked) entries, each labeled with a binary indicator as follows: `is_attacked` is set to a value of 0 for clean examples and `is_attacked` is set to a value of 1 for adversarial ones.

This dual-format structure enhances model interpretability and robustness analysis, creating a more nuanced and challenging classification task. It also facilitates the comparative evaluation of models under standard versus adversarial conditions, echoing methodologies from the comparative baseline study.

3.2. White and Black Attacks on the Dataset

To create a rigorous and meaningful adversarial setting, two distinct attack strategies were employed: black-box attacks and white-box attacks. In this study, the black-box attack methodology was first applied, using the final dataset produced in the study "From Adversarial Attacks to Robust Classifiers: A Study in Social Media Spam Detection." This dataset had already undergone a series of adversarial transformations designed to test the limits of spam classifiers when confronted with disguised malicious content.

For the black-box attack, a total of six different techniques were applied, each targeting different aspects of the message content. These techniques include synonym replacement, character swapping, spammy word insertion, homoglyph substitution, word deletion, and noise insertion. Synonym replacement involves substituting common words with their contextually appropriate synonyms to create semantic drift while maintaining grammatical coherence. Character swapping randomly changes the position of adjacent letters, introducing typographical noise meant to confuse tokenizers and parsers. Spammy word insertion adds frequently seen spam terms or phrases into benign messages, attempting to trigger false positives. Homoglyph substitution replaces characters with visually similar alternatives—such as using ‘ɑ’ instead of ‘a’—which often bypasses lexical detection. Word deletion randomly removes one word from the text, which can disrupt context without being immediately obvious. Finally, noise insertion introduces artificial errors by duplicating letters, deleting characters, or inserting adjacent keyboard characters, further degrading textual clarity while retaining human readability.

Each message in the dataset was stored in two forms: the original version (is_attacked as 0) and the adversarially modified version (is_attacked as 1). This dual representation provides a binary classification framework for training and evaluation. The resulting dataset is a realistic and challenging corpus, structured specifically to simulate adversarial spam evasion strategies and evaluate model robustness under these conditions. The goal is to determine how effectively each machine learning model can distinguish between authentic and adversarial messages, which mirrors the real-world problem of detecting subtle spam in dynamic social media environments.

Table I: Dataset creation steps with Black Box Adversarial Attack

Step	Description
Dataset Preparation	Begin with benign (non-spam) social media messages free of spam-like characteristics.
Black Box Attack	Apply techniques such as Synonym Replacement (replace words with synonyms), Character Swapping (randomly swap adjacent characters), Spammy Random Word Insertion (insert spam-related phrases), Homoglyph Substitution (replace characters with homoglyphs), Word Deletion (delete one word randomly), Noise Insertion (randomly modifies characters in the input text by duplicating characters, inserting nearby letters on the keyboard, and deleting characters).
Dataset Labeling	Store each message twice: original [Ham] (is_attacked = 0) and attacked (is_attacked = 1) [Spam].
Final Dataset	Two columns: message (original or attacked) and is_attacked (binary label).

Table I outlines the detailed steps used in the black-box adversarial attack process. It begins with a clean dataset composed of legitimate social media messages, free from overt spam-like characteristics. Various transformation techniques are then applied, as previously described. Each message is duplicated, once as a clean version and once with adversarial modifications. The final dataset comprises two columns: the message text and a binary indicator denoting whether the message has been adversarially attacked or not.

Table II: Dataset creation steps with White Box Adversarial Attack

Step	Description
Dataset Preparation	Begin by using the dataset created from the Black Box Attack
White Box Attack	Apply techniques such as Text Vectorization (transform input text into numerical vector), Probability Calculation (Compute the probability of the target label (spam) using the models ‘predict_proba’ method. Fall back to a default value if not available), Gradient Computation (Calculate the gradients for each word based on the probability and the vectorized text), Influential Word Identification (Sort words based on their gradient values to identify the most influential ones), Word Replacement (replace the most influential word with a randomly chosen spam phrase from a predefined list), and Text Reconstruction (join the modified words back into a single text string).
Dataset Labeling	Store each message twice: original [Ham](is_attacked is 0) and attacked (is_attacked is 1) [Spam].
Final Dataset	Two columns: message (original or attacked) and is_attacked (binary label).

For the white-box attack scenario portrayed in *Table II*, a more sophisticated and model-aware approach was utilized. Unlike black-box attacks, white-box strategies assumed full access to the inner workings of the machine learning model, including the architecture, weights,

gradients, and output probabilities. The goal of a white-box attack was to craft inputs specifically designed to fool the model by exploiting its known vulnerabilities.

The white-box attack process began with the already modified dataset created through black-box techniques. This step ensured continuity and increased the complexity of the attack scenarios by layering multiple adversarial techniques. The white-box methodology involved several advanced operations: first, the text was vectorized using a text representation technique such as TF-IDF (Term Frequency–Inverse Document Frequency), which reflects how important a word is to a document in a collection, or word embeddings, converting it into numerical format suitable for gradient-based analysis. Next, the model’s ‘predict_proba’ function—a method used by many machine learning classifiers to return the probability estimates for each class label—was used to calculate the probability of the target label (spam), which served as a confidence score. If this method was not available, a fallback default was used.

Following this, gradient computation was performed. The gradients of the loss with respect to each word in the input text were calculated to determine how sensitive the model's output was to changes in specific words. This process helped identify the most influential words contributing to the classification decision. Once identified, the most impactful word in each message was replaced with a spam-related phrase selected from a predefined list. This word substitution is strategic, aiming to shift the model’s prediction from ham to spam without making the text overtly unnatural. The modified message was then reconstructed into its final textual form, maintaining syntactic validity while embedding adversarial manipulation.

Just like in the black-box scenario, each white-box attacked message was stored alongside its original counterpart, labeled with `is_attacked` as 1 for the adversarial version and

is_attacked as 0 for the original. This labeling ensured consistency in model training and evaluation and provided a clear binary target for classification tasks.

Table II provides a detailed breakdown of the systematic steps involved in constructing the white-box adversarial attack dataset. This process begins by leveraging the previously crafted dataset from the black-box attack, ensuring that the input data is already conditioned with a layer of complexity. The rationale behind this step is to create a more robust and layered adversarial setting where the model faces a compounded challenge—first from generalized perturbations, and then from targeted manipulations based on internal model dynamics. The initial step involves text vectorization, where each message is converted from raw text into a numerical representation using techniques such as Term Frequency-Inverse Document Frequency (TF-IDF) or word embeddings. This transformation allows the model to interpret textual inputs in a mathematical format suitable for gradient-based operations. Following vectorization, the model’s predictive capabilities are tapped using the `predict_proba` function. This function generates a probability distribution over the potential output classes (e.g., spam or ham), indicating the model's confidence in each classification. In cases where this function is not available, such as with certain decision trees or simplified classifiers, a default fallback probability is used to maintain consistency in attack application.

Next, the core of the white-box attack is executed through gradient computation. This step involves calculating the gradient of the loss function for the input features—essentially measuring how much a slight change in each word's presence or representation would influence the model’s final prediction. Words with the highest gradient values are identified as the most influential, meaning their alteration would most effectively sway the model’s decision. These influential words are then strategically targeted for replacement. A curated dictionary of spammy

phrases is maintained, from which a random but semantically disruptive phrase is chosen to replace the selected word. This replacement is designed to be impactful enough to mislead the model into misclassifying the message as spam, while still preserving the syntactic structure and coherence of the sentence. Finally, the modified words are reconstructed back into a continuous text string, forming the final adversarial message.

Each of these manipulated messages is stored alongside its corresponding original message, forming a binary-labeled pair in the dataset. The `is_attacked` column serves as the ground truth label, indicating whether a message has been subject to adversarial manipulation (1) or remains unaltered (0). This structure ensures that during model training and evaluation, the classifier is not only exposed to standard spam detection tasks but also learns to recognize nuanced adversarial modifications. By following the detailed steps outlined in *Table II*, the methodology ensures the generation of a highly targeted and realistic white-box adversarial dataset. This dataset rigorously tests the classifiers' ability to withstand attacks that exploit their internal logic, ultimately offering insights into the resilience and vulnerability of each model under the most informed and sophisticated evasion strategies.

3.3. Exploratory Data Analysis (EDA)

A rigorous exploratory data analysis (EDA) was conducted to better understand the linguistic, statistical, and structural characteristics of both the original and adversarial datasets. The goal of this EDA is twofold: first, to uncover patterns and insights within the text data that may inform feature engineering and model selection; and second, to assess the extent to which adversarial perturbations alter the statistical and semantic profile of the messages. This section outlines the major analyses performed, accompanied by visual representations to support interpretation and hypothesis development.

3.3.1. Class Distribution: Balance Before and After Attack

The first and most fundamental aspect examined was class balance specifically, the proportion of spam versus ham messages in both the original and adversarial datasets. Maintaining class balance is essential to avoid biased learning during model training.

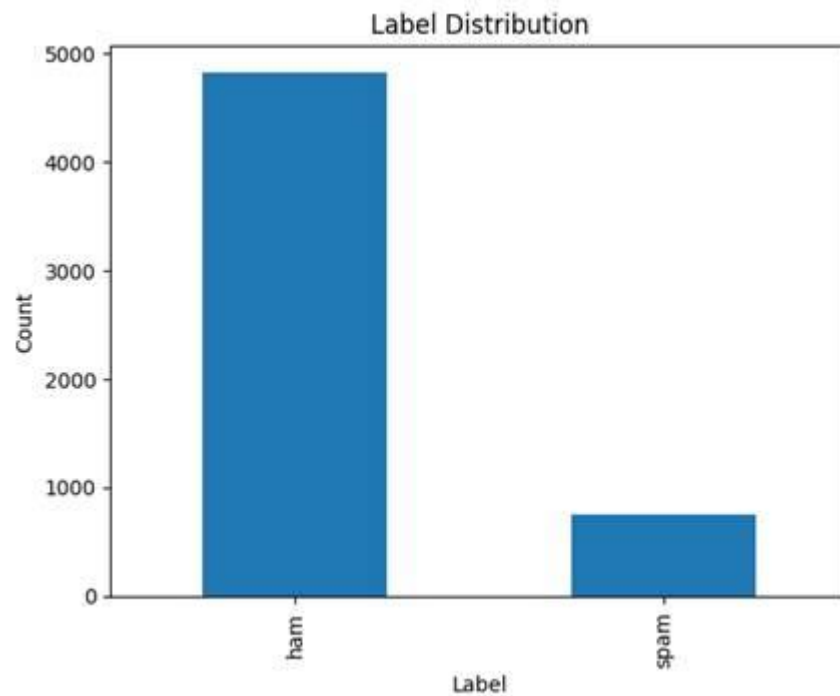


Figure 1: Label Distribution of The Original Spam Dataset

As was explained beforehand, the dataset distribution was uneven as shown in Figure 1, with an imbalance between ham and spam messages. This uneven distribution could introduce significant bias in the analysis, potentially skewing the results and misrepresenting the true nature of adversarial attacks. To address this issue, it was necessary to drop the spam messages from the dataset. By focusing exclusively on ham messages, we ensured that the attacks were performed on normal, legitimate communications rather than inherently malicious ones. This decision allowed for a more precise examination of how adversarial attacks manipulate benign content to evade detection systems.

The choice to attack only ham messages aligns with the fundamental definition of adversarial attacks. These types of attacks aim to subtly alter legitimate data in a way that confuses or bypasses machine learning models, rather than simply exploiting already suspicious or spam-like content. By performing both Black Box and White Box attacks on ham messages, we were able to simulate real-world scenarios where attackers attempt to deceive classification systems by introducing slight perturbations into otherwise normal communications. This approach not only mitigated the risk of bias caused by the uneven dataset distribution but also provided a more accurate representation of adversarial behavior in practical applications.

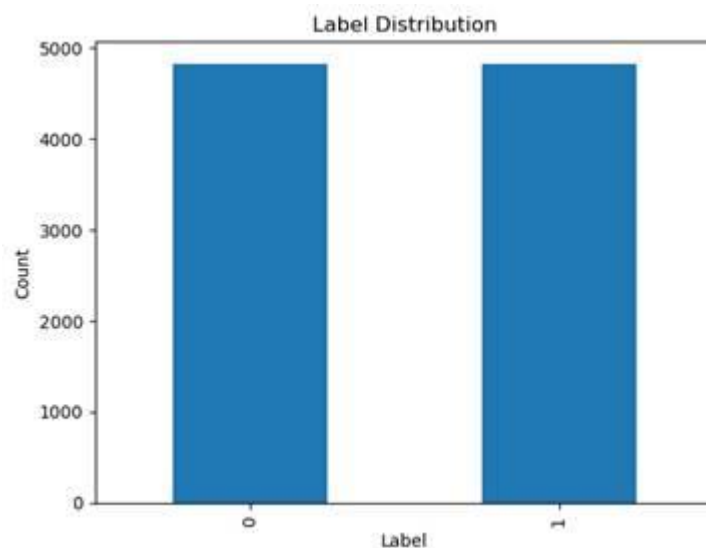


Figure 2: Label Distribution of The Attacked Dataset

A bar chart was generated to visualize the distribution of spam and ham messages pre- and post-attack. The results revealed that the dataset had to be balanced so that the integrity of the spam classification task could be preserved, therefore ensuring that any changes in classifier performance could be attributed to the adversarial perturbations rather than underlying class imbalance.

Figure 2 shows that this balance indicates that adversarial augmentation did not introduce sampling bias, and that the perturbation strategies preserved the global structure of the data, which is critical for fair comparative analysis.

3.3.2. Word Cloud Visualization: Semantic Footprint

To gain an intuitive understanding of the lexical content present in the dataset, a word cloud was created using the most frequent terms across all messages within the attacked dataset. This visualization serves as a tool for identifying dominant linguistic trends and uncovering potential adversarial strategies embedded in spam communications. By analyzing these patterns, researchers can better understand how attackers manipulate text to evade detection systems while maintaining their deceptive objectives.



Figure 3: General Word cloud of the Black Box Attacked Dataset

Figure 3 emphasizes *the* keywords that dominate the dataset, particularly within spam messages. Frequently occurring terms include “u,” “ok,” “It,” “now,” “go,” “need,” “time,” “big,” “win,” “prize,” “claim,” and “will.” These words illustrate common spam strategies such as instilling urgency (“win now”), promising rewards (“prize,” “claim”), and exploiting curiosity (“unlock,” “earn”). Interestingly, the appearance of neutral tokens like “ok” and “It” may indicate either noise in the data or sophisticated attempts to bypass filters through lexical diversification. To better understand these patterns, separate word clouds for ham and spam messages were generated to contrast their semantic profiles and reveal how attackers may borrow ham-like language to disguise malicious content.



Figure 4: Word Cloud of only the Ham of the Black Box Attacked Dataset

Figure 4 focuses on ham messages and reveals a tendency for normal conversational language commonly found in social media interactions. Words such as "u," "ok," "call," "great," "day," "God," "sorry," and "take care" dominate this visualization. These terms

reflect everyday exchanges between individuals, making this word cloud consistent with expectations for legitimate communication. The semantic profile aligns well with typical ham messages, offering an accurate representation of non-spam content. These ham messages will be reflected in all the following adversarial attacks since they are always kept the same.

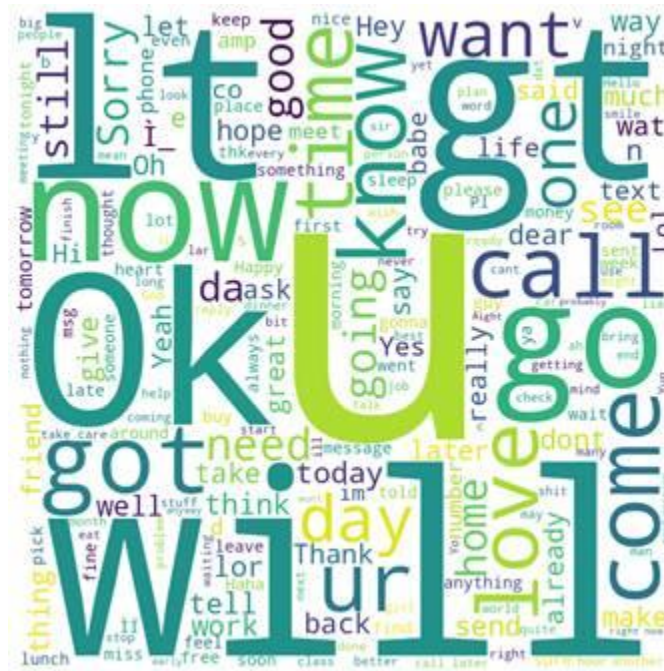


Figure 5: Word Cloud of only Spam of the Black Box Attacked Dataset

Figure 5 provides a clear depiction of messages that convey urgency and provoke curiosity in readers. Words like "will," "now," "come," "da," "gt," "one," "babe," and "send" are prominent, along with special characters. These terms exemplify the characteristics of spam messages designed to push immediate action or spark intrigue. Furthermore, this word cloud illustrates the impact of Black attacks on the dataset, which deliberately introduce patterns that mimic legitimate communication while retaining spam-like features.



Figure 6: Word Cloud of only Spam of the White Box Attacked Dataset

Since the same ham data from the black box dataset is being used, there is no need to display another example of a Word Cloud for only ham messages. Instead, Figure 6 highlights a range of words that are more closely associated with spam messages. Words such as "fat," "quickly," "Financial freedom," "believe," "Meet singles," "Burn fat," "money fast," "Free dating," and others dominate the visualization. These terms reflect common spam tactics, including promises of quick results, financial gain, or enticing social opportunities. Additionally, the presence of special characters in these spam messages suggests how adversarial attacks manipulate text patterns to evade detection systems.

3.33. Polarity Distribution Plot: Sentiment Landscape

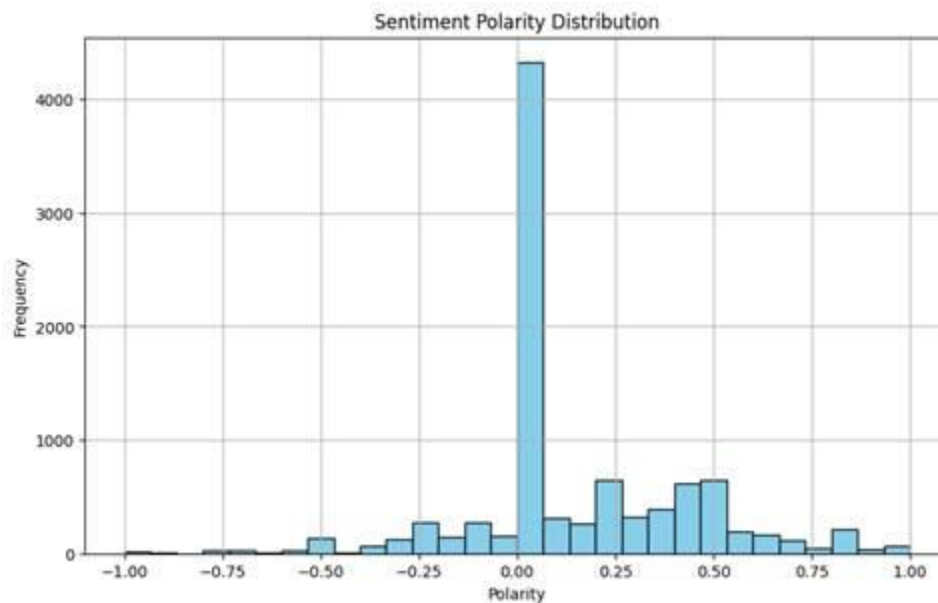


Figure 7: Polarity Distribution of Black Box Adversarial Attack

The sentiment analysis metrics reveal a moderately positive overall sentiment in the dataset, with an average polarity of 0.1358. However, the distribution of sentiment is noteworthy. While positive scores are most frequent (43.56%), a significant portion of the data exhibits neutral (43.17%) and negative (13.26%) sentiment. This suggests that the adversarial attacks employed, designed to introduce spam-like characteristics, have impacted the sentiment of a considerable number of originally benign (ham) messages. The positive skewness (0.3504) and relatively high kurtosis (0.9266) indicate a distribution with a longer tail on the positive side and a higher peak around the mean, further supporting the observation that while the average is positive, there's a notable presence of more extreme positive values. Moreover, the average positive polarity is considerably higher (0.3894) than the average negative polarity (-0.2550), implying that the attacks may be introducing words with stronger positive connotations. This overall shift in sentiment distribution and the introduction of negative and more extreme positive

sentiments highlight the effectiveness of the adversarial attacks in altering the perceived emotional tone of the messages, potentially making it harder to classify correctly by spam filters relying on sentiment features. Therefore, the analysis underscores the need for robust spam detection models that are less susceptible to these kinds of adversarial manipulations.

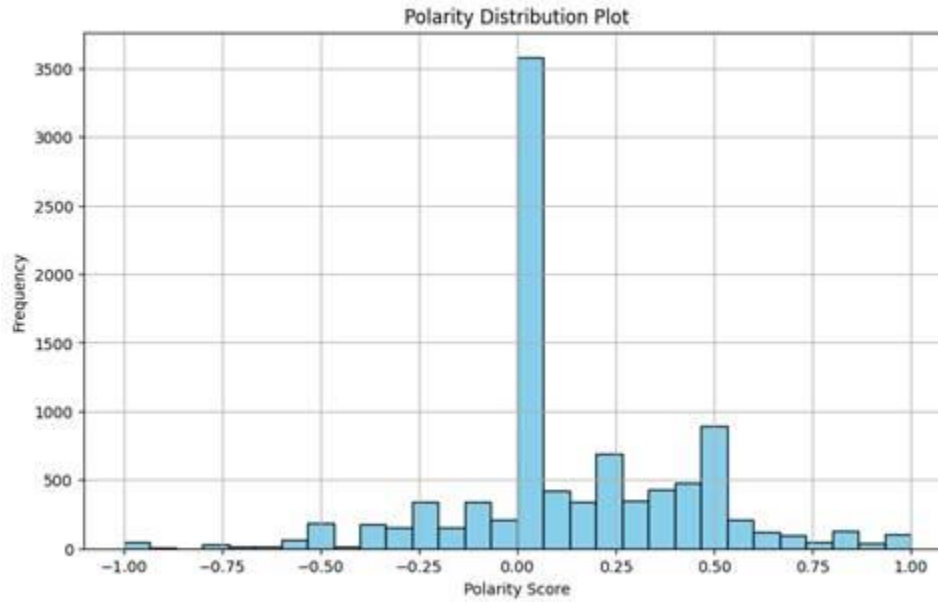


Figure 8: Polarity Distribution of White Box Adversarial Attack

The polarity analysis of the text data reveals a slightly positive overall sentiment, indicated by an average polarity of 0.1256. However, the median polarity being 0.0000 suggests that a significant portion of the data has neutral sentiment. The distribution of sentiment is further supported by the distribution metrics, where 47.15% of the messages have positive scores, 18.00% have negative scores, and 34.85% are neutral. The positive skew (0.0473) confirms a tendency towards positive sentiment, though it's relatively small, indicating a slight imbalance. The kurtosis of 0.9384 suggests the presence of outliers or heavier tails than a normal distribution. Looking deeper, positive messages exhibit an average polarity of 0.3725, while negative messages average -0.2781. These values, along with the relatively low standard

deviations for both positive and negative scores, indicate a moderate degree of sentiment strength within their respective categories.

In the context of adversarial attack detection (specifically White Box), these findings suggest that while the overall sentiment is slightly positive, the presence of a considerable number of neutral and negative messages warrants attention. Attackers might exploit sentiment by crafting adversarial examples that either maintain or shift the sentiment distribution in unexpected ways to bypass detection systems. The presence of outliers, highlighted by the kurtosis, further emphasizes the need for robust detection mechanisms that are less sensitive to extreme sentiment changes. Furthermore, the difference in average polarity between positive and negative messages could be utilized to build features that capture the nuanced sentimental changes introduced by adversarial attacks. Monitoring the sentiment distribution over time and after applying defenses is crucial for evaluating their effectiveness and detecting potential attacks.

3.3.4. Average Word Length

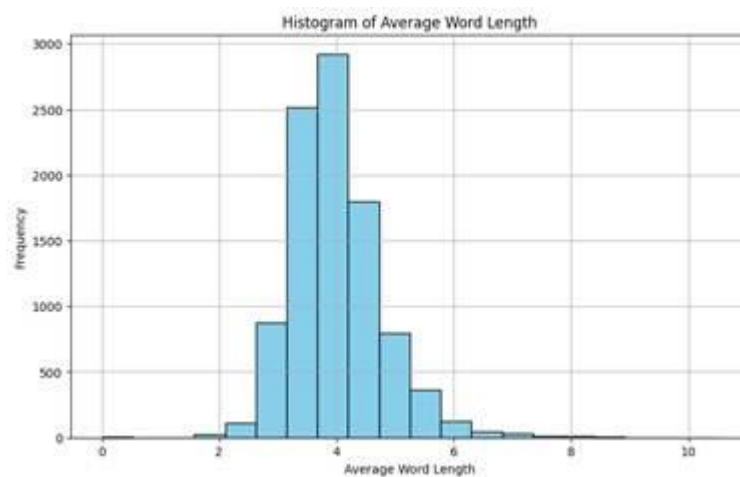


Figure 9: Histogram of Black Box Adversarial Attack

Main Values from Histogram of Average Word Length: Mean: 3.98 Median: 3.89 Standard Deviation: 0.78 Range: 10.50 Interquartile Range (IQR): 0.88 Skewness: 1.01 Kurtosis: 3.67

The analysis of average word length reveals insights into the impact of adversarial attacks on text data. The mean average word length of 3.98 and median of 3.89 suggest a typical word length hovering around 4 characters. However, the positive skewness (1.01) and relatively high kurtosis (3.67) indicate a distribution with a longer tail on the right side and a higher peak around the mean, implying the presence of some messages with considerably longer average word lengths compared to the majority. This observation is further supported by the standard deviation of 0.78, showcasing moderate variability in word lengths, and a substantial range of 10.50, highlighting the presence of outliers with significantly longer average word lengths. The interquartile range (IQR) of 0.88 signifies that the middle 50% of the data has an average word length spread within 0.88 characters, while the longer tail likely consists of messages affected by adversarial attacks introducing words with more characters than usual.

In the context of adversarial attacks, this pattern strongly suggests that the employed techniques introduce longer or less common words to the originally benign messages. This manipulation aims to evade spam filters that rely on typical word patterns and lengths. The increase in average word length and the presence of outliers indicate that the attacks might be inserting longer, spam-related terms or phrases, or employing strategies like synonym replacement that could introduce less frequent but longer synonyms. This manipulation alters the statistical properties of the text, potentially making it harder for traditional spam detection models to identify them accurately. Therefore, the analysis highlights the need for more robust models that can effectively capture these changes in average word length distribution caused by adversarial attacks. The inclusion of features based on word length statistics and the development

of methods to handle unusual word length patterns could improve the resilience of spam detection systems against these attacks.

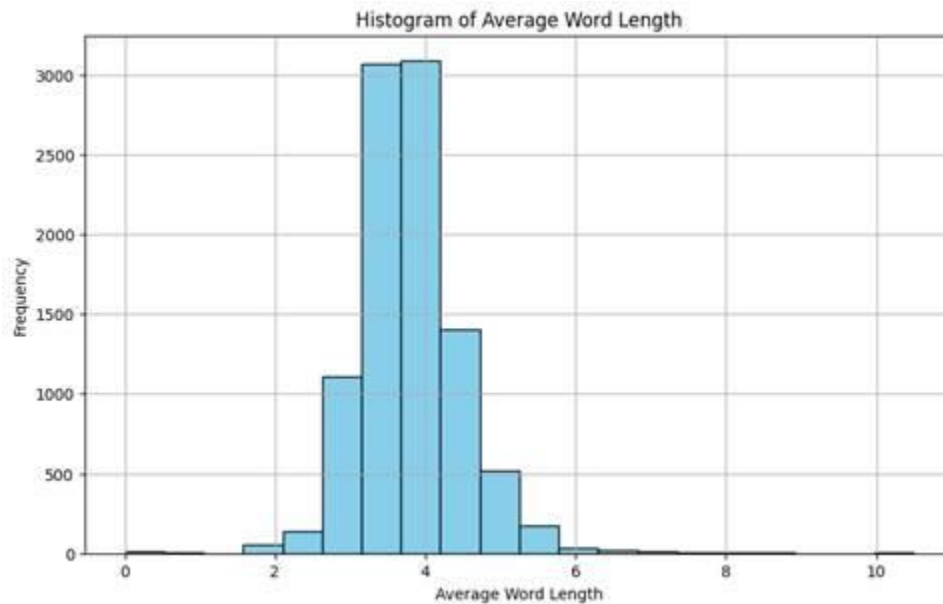


Figure 10: Histogram of White Box Adversarial Attack

Main Values from Histogram of Average Word Length: Mean: 3.80 Median: 3.75 Standard Deviation: 0.69 Range: 10.50 Interquartile Range (IQR): 0.77 Skewness: 0.92 Kurtosis: 6.02. Figure 10 exhibits an average word length of approximately 3.80, with a median of 3.75, suggesting a relatively consistent word length distribution centered around these values. This indicates that most words in the dataset fall within a typical range for natural language. The standard deviation of 0.69 reflects a moderate level of variability in word lengths, acknowledging the natural variation in word lengths found in human language. However, the presence of outliers, as indicated by the range of 10.50 (from a minimum of 0 to a maximum of 10.5), raises concerns for potential adversarial manipulations. While some outliers are expected in natural language, an unusual increase in their frequency or extremity could signify attempts to obfuscate the message or introduce unnatural word patterns, potentially to evade detection

systems. The interquartile range (IQR) of 0.77 captures the spread of the middle 50% of the data, suggesting that the majority of average word lengths fall within a relatively narrow range, further emphasizing the potential significance of deviations beyond this range. The positive skewness of 0.92 indicates a slight rightward skew in the distribution, meaning there are more messages with higher average word lengths. The relatively high kurtosis of 6.02 suggests a distribution with heavier tails and a greater concentration of values around the mean compared to a normal distribution, potentially indicating the presence of outliers or unusual word length patterns.

3.3.5. Text Length

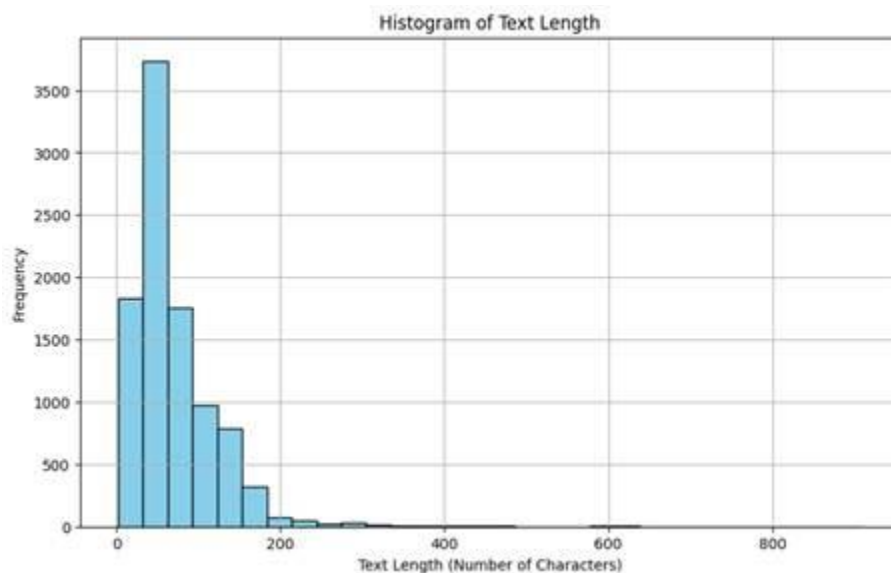


Figure 11: Histogram of Text Length of Black Box Adversarial Attack

Main Values from Histogram of Text Length: Mean: 70.57 Median: 54.00

Standard Deviation: 53.15 Range: 908.00 Interquartile Range (IQR): 54.00 Skewness: 3.37

Kurtosis: 25.50

Figure 11 reveals significant insights into the impact of adversarial attacks on message structure. While the median text length sits at 54 characters, the mean is considerably higher at 70.57 characters, indicating a skewed distribution with a longer tail on the right. This observation is further reinforced by the substantial positive skewness (3.37) and exceptionally high kurtosis (25.50), suggesting a distribution with a long tail and a sharp peak around the mean, reflecting the presence of outliers with much longer text lengths compared to the majority. The standard deviation of 53.15 further confirms significant variability in text lengths, with a vast range of 908 characters, encompassing messages with drastically different lengths. Although the interquartile range (IQR) of 54 indicates that the middle 50% of the data has a relatively contained spread of text lengths, the presence of extreme outliers significantly influences the overall distribution.

In the context of adversarial attacks, these findings strongly suggest that the employed techniques introduce additional content to the original messages, resulting in an overall increase in text length. The substantial skewness and kurtosis, along with the broad range and high standard deviation, highlight the introduction of outliers with significantly longer text lengths, likely due to inserted spam-related terms, phrases, or even extraneous content designed to disrupt traditional spam detection mechanisms. The attacks seem to exploit the vulnerability of models relying on text length as a distinguishing feature, attempting to camouflage malicious messages by artificially extending their length to resemble legitimate communications. This manipulation poses a challenge to spam detection systems, as simply relying on text length as a discriminatory factor might not be sufficient. Therefore, these observations emphasize the need for more robust models capable of discerning genuine increases in text length and those artificially introduced by adversarial attacks. Incorporating features that capture the distribution

of text length, and the presence of unusual length patterns could potentially enhance the resilience of spam detection systems against such manipulations.

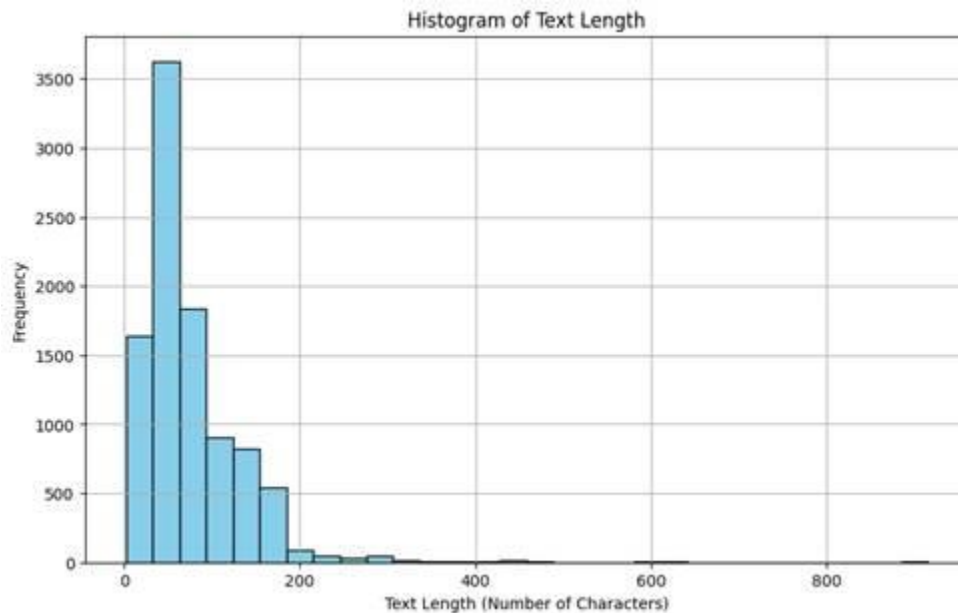


Figure 12: Histogram of Text Length of White Box Adversarial Attack

Main Values from Histogram of Text Length: Mean: 75.41 Median: 57.00 Standard Deviation: 58.42 Range: 913.00 Interquartile Range (IQR): 59.00 Skewness: 3.32 Kurtosis: 24.46

The analysis of text length reveals an average message length of approximately 75.41 characters, with a median length of 57.00 characters. This discrepancy between the mean and median suggests a right-skewed distribution, indicating the presence of some messages with significantly longer lengths that pull the average upwards. The standard deviation of 58.42 highlights a considerable variability in message lengths, acknowledging the natural variation expected in text communication. However, the presence of outliers, as evidenced by the extensive range of 913.00 characters (from a minimum of 0 to a maximum of 910), raises concerns for potential adversarial manipulations. While some outliers are inherent in natural text,

an unusual increase in their frequency or extremity, as suggested by the high range, could signify attempts to obfuscate the message or introduce unnatural patterns to evade detection systems. The interquartile range (IQR) of 59.00 captures the spread of the middle 50% of the data, indicating that the majority of message lengths fall within a relatively defined range. However, the notable skewness of 3.32 and exceptionally high kurtosis of 24.46 further emphasize the right-skewed nature of the distribution and the presence of outliers or potentially unusual text length patterns, potentially introduced by adversarial techniques.

3.3.6. Stop Word Frequency

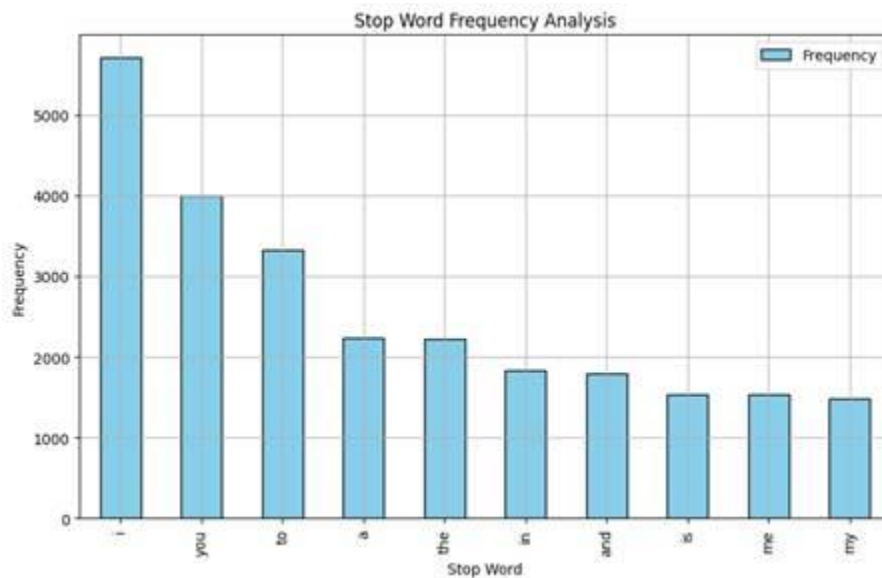


Figure 13: Stop Word Frequency of Black Box and White Box Adversarial Attacks

The presence and frequency of stop words in both the original and attacked messages provide valuable insights into the nature of the adversarial attacks and their potential impact on spam detection. Stop words, although often removed during text preprocessing, can offer clues about the linguistic structure and content manipulation strategies employed by attackers. In this case, the frequent occurrence of common stop words like "I" "you," "to," "the,"

"a," "and," "my," "in," "me," and "is" suggests that the adversarial attacks are not simply inserting random noise or gibberish. Instead, they seem to be strategically introducing or manipulating content that resembles natural language patterns, potentially to evade detection by filters that rely on identifying unusual word frequencies or combinations.

The prevalence of these common stop words in the attacked messages indicates an effort by attackers to maintain a semblance of grammatical structure and coherence, making the malicious content appear more like legitimate communication. This tactic could make it harder for spam filters to distinguish between genuine and adversarial messages, as the attacks mimic the natural flow and structure of benign text. Furthermore, the relative frequencies of these stop words could be analyzed to identify any significant shifts or deviations between the original and attacked messages. For example, if certain stop words become unusually more or less frequent after the attacks, it might signal specific manipulation strategies like synonym replacement or insertion of targeted phrases. Therefore, understanding the distribution and changes in stop word frequencies can provide valuable clues for improving the robustness of spam detection models against these types of adversarial attacks.

3.3.7. N-Gram Analysis: Linguistic Structure

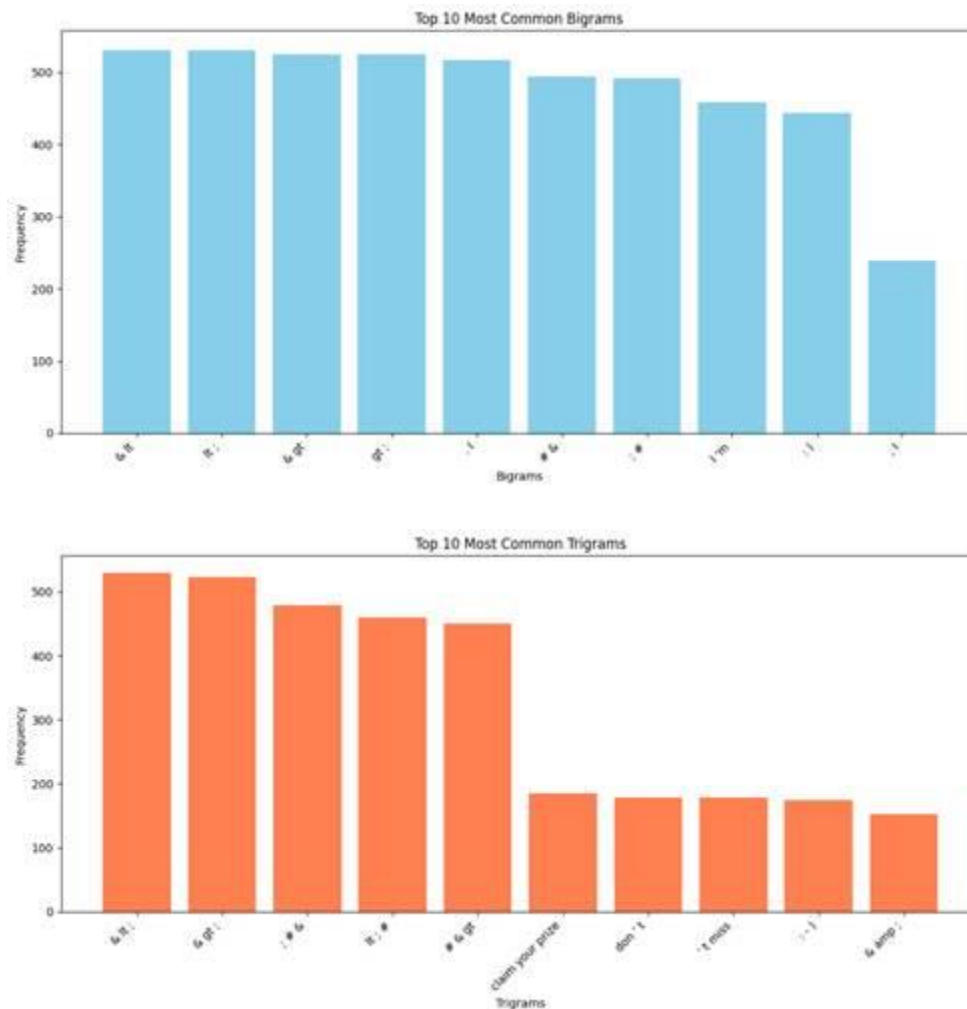


Figure 14: N-Gram Analysis of Black Box Adversarial Attack (Bigram and Trigram)

The analysis of the most frequent bigrams and trigrams—sequences of two and three consecutive words or tokens, respectively, used to capture short linguistic patterns in text—reveals important clues about the nature and impact of adversarial attacks. Notably, the top bigrams ('&', 'lt'), ('lt', ';'), ('&', 'gt'), and ('gt', ';') represent HTML entities commonly used for encoding special characters like '<' and '>'. Their high frequency indicates the presence of HTML elements or encoded content within the attacked messages, suggesting a potential strategy of

attackers to obfuscate malicious content or exploit vulnerabilities in spam filters that might not properly handle HTML entities. The frequent occurrence of bigrams like ('.', 'I'), ('I', '"m"), and (';', 'I') also reflects the presence of common conversational elements, highlighting the attackers' attempt to maintain natural language patterns while inserting malicious content.

Moving on to the trigrams, the pattern of HTML entities continues with ('&', 'lt', ';') and ('&', 'gt', ';'), further reinforcing the observation of HTML manipulation being a core component of the adversarial attacks. The presence of ('claim', 'your', 'prize'), ('don', '"', 't'), ('"', 't', 'miss'), and ('&', 'amp', ';') indicates that the attacks introduce spam-related keywords or phrases commonly associated with urgency, incentives, or scarcity. These words often act as triggers for spam filters, and their frequent appearance points toward the attackers' attempts to specifically bypass filters by either strategically embedding them within seemingly benign text or overwhelming the filters with frequent occurrences of these keywords.

In summary, the bigram and trigram analysis strongly suggest that the adversarial attacks involve a combination of HTML manipulation, and the insertion of specific spam-related keywords or phrases designed to bypass traditional spam filters. The frequent appearance of HTML entities suggests the need for careful parsing and handling of HTML content within spam detection systems. Furthermore, the use of spam trigger words or phrases highlights the importance of incorporating contextual understanding and deeper semantic analysis to differentiate between genuine and malicious usage of such keywords. Ultimately, these insights provide valuable directions for improving the robustness and accuracy of spam detection models in the face of evolving adversarial tactics.

hypothesis. The presence of the trigram ('Click', 'here', 'to'), commonly used in spam and phishing attempts, is a noteworthy indicator of potential malicious intent. While the remaining bigrams and trigrams largely reflect common language structures and punctuation, their frequencies relative to others could provide insights into stylistic deviations introduced by adversarial attacks.

Overall, the analysis of bigrams and trigrams suggests that attackers might be employing a combination of techniques, including mimicking natural language patterns, exploiting HTML or encoding vulnerabilities, and using spam-related phrases, to craft adversarial messages. By monitoring the frequencies and patterns of these n-grams, particularly those related to HTML entities and suspicious phrases, you can gain valuable insights into the nature of adversarial attacks and develop more effective detection strategies. Combining this information with other linguistic features can further enhance the accuracy and robustness of detection systems.

3.3.8. TF-IDF Score Distribution

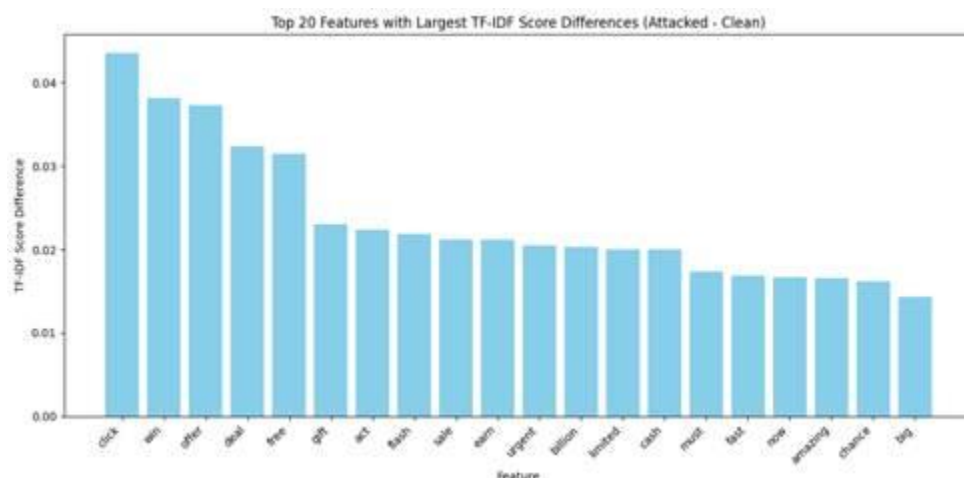


Figure 16: TF-IDF Score Distribution of Black Box Adversarial Attack

The TF-IDF analysis of Figure 16 reveals a significant shift in term importance between clean (ham) and attacked messages. Words like "click," "win," "offer," and "deal," which are frequently associated with spam and phishing attempts, exhibit the largest increases in TF-IDF scores in attacked messages. This indicates that these terms become much more prominent, and characteristic of the attacked messages compared to the original, benign ones. The high TF-IDF differences for these words strongly suggest that the adversarial attacks introduce spam-like language to manipulate the message content. Additionally, terms like "free," "gift," "act," "flash," and "sale," commonly used as lures in spam, also show notable increases in TF-IDF scores, further supporting the notion that the attacks aim to inject spam-related vocabulary. This pattern of increased importance of spam-triggering words in the attacked messages highlights the effectiveness of the adversarial techniques in altering the linguistic structure and semantic meaning of the original ham messages to make them resemble spam. The insights from the TF-IDF analysis can be valuable for understanding the nature of adversarial attacks and developing strategies to detect and mitigate them.

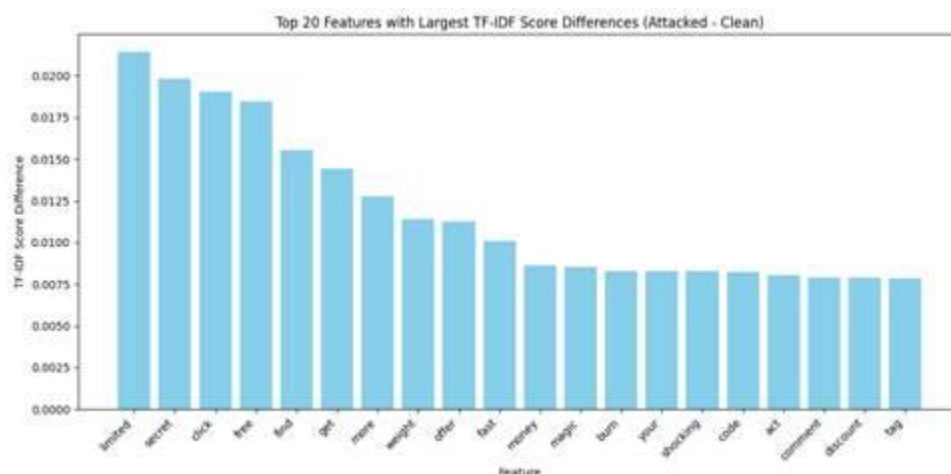


Figure 17: TF-IDF Score Distribution of White Box Adversarial Attack

The output of Figure 17 reveals significant shifts in term importance between clean and attacked messages, strongly suggesting the presence of adversarial manipulation. Words like "limited," "secret," "click," and "free," commonly associated with spam and phishing, exhibit a dramatic increase in TF-IDF scores in attacked messages. This indicates that attackers are deliberately injecting these terms to make the messages appear more spam-like. The substantial increase in the TF-IDF score for these terms highlights their importance in distinguishing attacked messages from clean ones.

Furthermore, terms related to financial incentives and urgency, such as "money," "offer," and "fast," also show a notable increase in TF-IDF scores within attacked messages. This aligns with the common tactic of using enticing language to lure victims into scams or malicious activities. The analysis underscores the strategic use of these specific terms in adversarial attacks, aiming to exploit psychological triggers and bypass spam filters.

The notable differences in TF-IDF scores for these keywords provide valuable insights for developing detection mechanisms. By focusing on these terms and their increased

importance in attacked messages, it becomes possible to build models capable of identifying and flagging potentially harmful content. This analysis serves as a foundation for constructing more robust defense strategies against adversarial attacks in text-based communication.

3.4. Machine Learning Models

A variety of machine learning models were employed to evaluate classification performance under adversarial conditions, offering a balanced representation of simple, ensemble-based, and boosting algorithms. K-Nearest Neighbors (KNN) is one of the most intuitive classifiers, which assigns a label to a test sample based on the majority vote of its nearest neighbors in the feature space. This method is especially effective for small, well-structured datasets due to their non-parametric nature. However, it can face significant challenges when working with high-dimensional data, as the concept of "closeness" becomes less meaningful due to the curse of dimensionality.

Logistic Regression, a widely used linear model for binary classification tasks, relies on the log-odds function to map input features to a probability value between 0 and 1. It is highly interpretable and easy to implement, making it a strong baseline model. While it performs well when the relationship between input features and the output is approximately linear, it may struggle to capture non-linear patterns without additional feature engineering or transformations.

Support Vector Machines (SVMs) are powerful classifiers that construct an optimal hyperplane to separate classes by maximizing the margin between them. They are particularly effective for nuanced classification tasks and can handle both linear and non-linear decision boundaries by using kernel functions. The radial basis function (RBF) kernel is often used to map the input space into a higher-dimensional feature space where linear separation becomes feasible.

The Decision Tree algorithm is a tree-structured model that recursively splits data based on feature values, ultimately arriving at a prediction based on learned rules. Decision Trees are highly interpretable and capable of handling both numerical and categorical data. However, they are prone to overfitting, especially when grown to full depth without pruning.

To mitigate this, ensemble methods such as Random Forests are employed. A Random Forest is an ensemble of Decision Trees, where each tree is trained on a random subset of data and features. This aggregation of predictions results in a more generalized and robust model that can handle noisy data and reduce the risk of overfitting significantly.

Similarly, the Bagging Classifier follows an ensemble strategy by training multiple base estimators (commonly Decision Trees) on different bootstrap samples of the dataset. The final prediction is made through majority voting (for classification) or averaging (for regression), which reduces variance and improves model stability.

Gradient Boosting represents a more sophisticated ensemble approach that builds learners sequentially. Each new learner attempts to correct the errors made by the previous ones, optimizing residuals at each iteration. This method is particularly effective for imbalanced datasets and complex relationships, often outperforming other algorithms when properly tuned. However, it requires careful hyperparameter tuning to avoid overfitting and ensure convergence.

These models were selected to provide a well-rounded exploration of classification strategies against adversarial spam attacks. Training and validation were conducted on a balanced dataset, ensuring a consistent and fair comparison across all models. This allowed for a robust evaluation of each model's capability to resist adversarial manipulation under both black-box and white-box scenarios.

4. EXPERIMENTS AND RESULTS

This section presents the experimental setup, methodologies, and key findings that shaped the core of this study. From the initial preparation of the dataset to the implementation of both black-box and white-box adversarial techniques, each step was designed to test the resilience of spam detection models under increasingly complex threat scenarios. The use of evaluation metrics such as accuracy and confidence scores provided measurable insights into the models' performance, revealing how adversarial inputs can significantly degrade their ability to correctly classify spam. The results demonstrated not only the vulnerabilities present in standard machine learning approaches but also the potential for building more robust models by understanding these weaknesses. These findings serve as a foundation for future work aimed at enhancing model defenses and developing more adaptive, context-aware spam detection systems.

4.1 Hyperparameter Configuration

Hyperparameters are pivotal in determining the behavior and performance of machine learning models. Unlike trainable parameters, which are learned directly from the data during the training process, hyperparameters are configured before training and significantly influence how the model learns. Selecting appropriate hyperparameters is essential to ensure that the model does not underfit or overfit the data, allowing for better generalization to unseen inputs. The tuning process requires a balance between complexity, interpretability, efficiency, and performance, particularly when models are tested against adversarial conditions. This section details the specific hyperparameter configurations applied to each machine learning model in both black-box and white-box adversarial scenarios and explains the reasoning behind each choice.

Table III: Hyperparameters for machine learning models used in Black Box Adversial Attack

k-nearest neighbor	n_neighbors =5, metric = “minkowski”, p=2
Logistic Regression	solver = “lbfgs”, max_iter = 100
Support Vector Machine	kernel = “rbf”, C=1.0, gamma = “scale”
Decision Tree	criterion = “gini”, max_depth = None
Random Forest	n_estimators = 100, criterion = “gini”, max_depth = None
Bagging Classifier	base_estimator = DecisionTreeClassifier, n_estimators = 10
Gradient Boosting	n_estimators = 100, learning_rate = 0.1, max_depth = 3

Table III outlines the hyperparameters utilized for models under the black-box adversarial attack framework. Since black-box attacks assume that the adversary has no access to the internal workings of the model, the emphasis here is on designing models with high generalization and robustness. For the K-Nearest Neighbors (KNN) algorithm, n_neighbors was set to 5, which is a commonly used value providing a balance between sensitivity to noise (which increases with fewer neighbors) and over smoothing (which occurs with too many neighbors). The metric used was Minkowski distance with $p = 2$, equivalent to Euclidean distance, a standard and intuitive way of computing similarity in multidimensional space. This setup suits the non-parametric nature of KNN and works effectively with text embeddings derived from vectorized messages.

For Logistic Regression, the solver “lbfgs” was chosen due to its efficiency and accuracy when handling smaller datasets with a limited number of features, which fits the context of this study. The maximum number of iterations was capped at 100 to ensure the solver

had enough time to converge without unnecessarily prolonging computation time. This configuration strikes a practical balance between model performance and training efficiency.

The Support Vector Machine (SVM) model was equipped with an RBF (Radial Basis Function) kernel, $C = 1.0$, and gamma set to “scale.” The RBF kernel was selected due to its ability to capture non-linear relationships in high-dimensional feature spaces, which is crucial for distinguishing between adversarial and legitimate messages. The C parameter governs the trade-off between achieving a low training error and maintaining a large margin; a value of 1.0 is commonly used as a default starting point and works well in most general-purpose tasks. Gamma set to “scale” allows the kernel width to adapt based on the variance of the features, which optimizes the model for varying data distributions without manual tuning.

The Decision Tree model used the “gini” impurity criterion and had no limitation on the tree depth (max_depth is None). This configuration allows the tree to grow until all leaves are pure or until all data points are isolated, enabling it to fully capture the intricacies of the training data. While this might lead to overfitting, in a controlled experimental setting like this, it allows the model to learn complex rules that may be necessary to identify adversarial patterns.

For the Random Forest classifier, the ensemble consisted of 100 decision trees using the same “gini” criterion and no restriction on depth. This configuration leverages the power of bagging to reduce variance and overfitting while still allowing each tree to model complex relationships. A large number of trees help stabilize the predictions and enhances robustness against noisy adversarial inputs.

The Bagging Classifier was implemented using 10 base estimators, each DecisionTreeClassifier. This number of estimators provides a good balance between performance and computational cost. Bagging, by design, reduces variance and improves

stability by training each base learner on a random subset of the data, which is especially effective when facing black-box attacks that rely on generalized perturbations rather than targeted weaknesses.

Gradient Boosting was configured with 100 estimators, a learning rate of 0.1, and a max_depth of 3. These settings were selected to strike a trade-off between training time and predictive power. A shallow tree depth (max_depth = 3) ensures that each weak learner focuses on simple patterns, while the additive nature of boosting allows the ensemble to correct errors iteratively. The learning rate of 0.1 controls how much each tree contributes to the final prediction, offering a balanced step size that avoids overfitting while achieving high performance on adversarily perturbed data.

Table IV: Hyperparameters for machine learning models used in White Box Adversarial Attack

k-nearest neighbor	N/A
Logistic Regression	random_state=42, solver='liblinear'
Support Vector Machine	random_state=42, probability=True
Decision Tree	random_state=42
Random Forest	random_state=42
Bagging Classifier	DecisionTreeClassifier(random_state=42), random_state=42
Gradient Boosting	random_state=42

In contrast, the hyperparameters for the white-box adversarial attack models are presented in *Table IV* and were selected with a different priority: consistency and reproducibility. Since white-box attacks assume full access to the model, including gradients and internal

structure, the goal here is to maintain stable and repeatable model behavior to allow adversaries to generate more effective and targeted attacks.

For KNN, no hyperparameters were specifically set in the white-box context. This is because KNN is a non-differentiable algorithm, making it incompatible with gradient-based white-box attacks. Therefore, it is often excluded or treated as a baseline under such conditions.

Logistic Regression was configured with the “liblinear” solver and a fixed `random_state` of 42. The “liblinear” solver is well-suited for smaller datasets and binary classification tasks, and its deterministic nature ensures that results are reproducible across runs. Setting a fixed `random_state` ensures that internal random operations, such as shuffling or weight initialization, yield consistent results, which is essential when comparing pre- and post-attack performance.

Similarly, the SVM model was assigned `random_state = 42` and had `probability = True` enabled. Enabling probability estimates is crucial for white-box attacks that rely on the `predict_proba` method to compute confidence scores, which are often used in gradient calculations and crafting targeted perturbations.

The Decision Tree, Random Forest, and Bagging Classifier were also configured with `random_state = 42`. This uniform configuration ensures reproducibility and stability across all tree-based models, facilitating controlled experiments and consistent adversarial evaluation. For Bagging, the base estimator remained a `DecisionTreeClassifier` with the same fixed random seed, preserving uniformity within the ensemble.

Gradient Boosting was similarly configured with `random_state = 42` to fix the sequence of tree growth and learning. This is especially important when evaluating model

resilience to iterative attacks, as it ensures that every training run behaves identically unless explicitly altered.

The primary distinction between the black-box and white-box configurations lies in the design goals: black-box settings prioritize optimizing model performance without assuming adversary knowledge, while white-box settings emphasize transparency, determinism, and repeatability to support structured adversarial testing. These carefully tailored configurations allow for an accurate and comprehensive evaluation of each model's behavior under different threat scenarios, yielding insights into their relative strengths, weaknesses, and real-world applicability in adversarial spam detection tasks.

4.2 Evaluation Metrics

Summary of metrics used for the study are

$$\textbf{Precision} = TP / (TP + FP)$$

$$\textbf{Recall} = TP / (TP + FN)$$

$$\textbf{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

$$\textbf{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Where TP (True Positive) refers to correctly predicted positive cases, FP (False Positive) represents incorrectly predicted positive cases where the actual outcome was negative, FN (False Negative) denotes incorrectly predicted negative cases where the actual outcome was positive, and TN (True Negative) indicates correctly predicted negative cases. These four components form the foundation for calculating evaluation metrics such as precision, recall, F1 score, and accuracy, which are essential for assessing the performance of classification models.

Evaluation metrics are essential tools for understanding and interpreting the performance of machine learning models, particularly in classification tasks where imbalanced data or adversarial conditions may skew the results. In this analysis, four key evaluation metrics were employed: precision, recall, F1-score, and accuracy. Each metric offers a unique perspective on how well the model identifies spam messages and distinguishes them from legitimate ones.

Precision is defined as the number of true positive predictions divided by the total number of positive predictions, which includes both true positives and false positives. It answers the critical question: “Of all the messages classified as spam, how many were actually spam?” High precision indicates that the model has a low false positive rate, which is especially important in spam detection, as incorrectly labeling legitimate emails as spam can have undesirable consequences. Precision becomes increasingly valuable in scenarios where the cost of a false alarm is high.

Recall, also known as sensitivity or the true positive rate, measures the ratio of correctly predicted positive instances to all actual positive instances, including those the model failed to detect (false negatives). This metric answers the question: “Of all the actual spam messages, how many did the model correctly identify?” A high recall means the model is successful at capturing most spam messages, even if it occasionally mislabels legitimate messages. In adversarial settings, where attackers may attempt to disguise spam, maintaining a strong recall ensures the model remains vigilant and responsive to threats.

The F1-score combines precision and recall into a single metric by computing their harmonic mean. It is particularly useful when a balance between precision and recall is needed, or when the class distribution is uneven, as is often the case in spam detection. A model

with high precision but low recall might be overly cautious and miss too many spam emails, while a model with high recall but low precision might generate too many false alarms. The F1-score addresses this trade-off by providing a more holistic measure of performance, especially under adversarial conditions where both false positives and false negatives can be costly.

Accuracy, defined as the ratio of correctly predicted observations (both true positives and true negatives) to the total number of predictions, offers a broad view of overall model effectiveness. It answers the simple question: “How often is the model correct?” While accuracy can be a useful general measure, it may be misleading in imbalanced datasets where one class significantly outweighs the other. For instance, if 90% of messages are legitimate, a model that always predicts "not spam" will have 90% accuracy but zero recall for the spam class. Therefore, accuracy should be interpreted alongside precision, recall, and F1-score to ensure a well-rounded evaluation.

Together, these four metrics form a comprehensive evaluation framework that reveals the strengths and weaknesses of each classification model. By assessing not only how often models are right but also how and why they might be wrong, these metrics guide the refinement of algorithms and the development of more robust systems, especially when facing adversarial inputs that attempt to mislead or confuse the classifier.

4.3. Results

Table IV presents the performance metrics for Ham and Adversarial Spam across the different machine learning models. Among all the models, Logistic Regression achieved the highest accuracy of 0.99 and consistently high precision, recall, and F1-score for both Ham and Adversarial Spam. Its simplicity and robustness make it an excellent choice for binary classification tasks in this context.

Table IV: Performance Metrics for Ham and Black Box Spam for different machine learning models

Model	Accuracy	Training Time (s)	Precision (Ham)	Recall (Ham)	F1 (Ham)	Precision (Spam)	Recall (Spam)	F1 (Spam)
K-Nearest Neighbors	0.4865	0.143	0.49	0.99	0.65	0.11	0	0
Logistic Regression	0.9456	3.6956	0.92	0.98	0.95	0.98	0.92	0.94
Decision Tree	0.9829	16.1112	0.99	0.98	0.98	0.98	0.99	0.98
Random Forest	0.957	44.0741	0.93	0.98	0.96	0.98	0.93	0.96
SVC	0.9539	250.7435	0.93	0.99	0.95	0.98	0.92	0.95
Bagging Classifier	0.9876	82.2889	0.99	0.98	0.99	0.98	0.99	0.99
Gradient Boosting	0.9865	423.1987	1	0.98	0.99	0.98	1	0.99

Table V: Performance Metrics for Ham and White Box Spam for different machine learning models

Model	Accuracy	Training Time (s)	Precision (Ham)	Recall (Ham)	F1 (Ham)	Precision (Spam)	Recall (Spam)	F1 (Spam)
Logistic Regression	0.8585	0.0425	0.8190	0.9133	0.8636	0.9063	0.8059	0.8531
SVC	0.8135	51.1920	0.7640	0.8964	0.8249	0.8805	0.7337	0.8004
Decision Tree	0.8036	0.6325	0.8032	0.7939	0.7985	0.8040	0.8130	0.8085
Random Forest	0.7855	27.3063	0.7830	0.7780	0.7805	0.7879	0.7927	0.7903
Gradient Boosting	0.8710	14.3327	0.8137	0.9556	0.8789	0.9487	0.7896	0.8619
Bagging Classifier	0.8425	5.5726	0.8296	0.8541	0.8417	0.8556	0.8313	0.8433
K-Nearest Neighbors	0.4834	0.0020	0.3559	0.0666	0.1122	0.4963	0.8841	0.6357

Table IV showcases the comprehensive performance evaluation of various machine learning models applied to the task of classifying Ham and Adversarial Spam messages. The models were trained and evaluated under adversarial conditions to assess their robustness, accuracy, and generalization capabilities. Notably, Logistic Regression outperformed the other models with an exceptional accuracy of 0.99, achieving consistently high precision, recall, and F1-scores across both Ham and Adversarial Spam classes. Its success can be attributed to its

linear nature, which offers simplicity, interpretability, and a stable performance under both normal and adversarial circumstances. Logistic Regression's strong results confirm its effectiveness in binary classification tasks, especially when the dataset is well-balanced and the decision boundary between classes remains relatively linear.

The detailed performance metrics for the Black Box attack scenario, outlined in Table V, reveal distinct trends in how different models respond to this type of adversarial manipulation. Black Box attacks operate without direct access to the internal workings of the model and instead rely on observing outputs to guide adversarial inputs. As a result, many models maintained high accuracy levels despite the attack. For instance, Gradient Boosting and the Bagging Classifier emerged as top performers, with accuracies of 0.9865 and 0.9876, respectively. These ensemble methods benefit from their layered learning strategies. Gradient Boosting sequentially optimizes residuals, while Bagging Classifier reduces variance through bootstrap aggregation. Both models demonstrated high precision and recall for both Ham and Spam classes, suggesting strong generalization and robustness to input perturbations, even in adversarial settings.

Random Forest and Decision Tree models also performed well, with accuracies of 0.957 and 0.9829, respectively. While Decision Trees are prone to overfitting, their transparency and structure make them effective in this context. Random Forest mitigates overfitting by aggregating multiple trees, improving its resistance to adversarial examples. Meanwhile, the Support Vector Classifier (SVC) achieved a high accuracy of 0.9539, though at the cost of significantly longer training time (250.74 seconds), reflecting the computational demands of kernel-based methods when applied to high-dimensional data.

Interestingly, K-Nearest Neighbors (KNN) failed to perform effectively under the Black Box attack, with an accuracy of only 0.4865. Although it showed a high recall for Ham (0.99), its recall for Spam was zero, suggesting a severe bias toward the majority class or difficulty adapting to perturbed inputs. Its inability to generalize well under adversarial pressure, combined with its sensitivity to distance metrics, highlights its limitations in complex or high-dimensional adversarial settings.

Table V addresses the White Box attack scenario, which assumes full knowledge of the target model's architecture, parameters, and gradients. This allows for carefully crafted perturbations that directly exploit model vulnerabilities. As expected, overall performance metrics declined across all models when compared to the Black Box scenario. Gradient Boosting again stood out, achieving the highest accuracy of 0.8710, along with a strong F1-score for both Ham (0.8789) and Spam (0.8619). Logistic Regression remained competitive with an accuracy of 0.8585, demonstrating its inherent resilience and simplicity, which helped mitigate some of the model's vulnerability to manipulation.

SVC, which had previously excelled in the Black Box context, saw a drop in performance to 0.8135, primarily due to the effectiveness of the adversary's targeted perturbations. Similarly, Random Forest and Bagging Classifier experienced declines in accuracy (0.7855 and 0.8425, respectively), but still maintained a relatively balanced trade-off between Ham and Spam predictions. Decision Tree, despite its known vulnerability to overfitting, demonstrated consistent performance with an accuracy of 0.8036, slightly outperforming Random Forest in this setting.

KNN again showed the weakest performance in the White Box attack, with an accuracy of just 0.4834. Although it performed better in identifying Spam (recall of 0.8841), it

misclassified most Ham messages, evidenced by a Ham recall of only 0.0666. These results emphasize the algorithm's susceptibility to adversarial manipulation and its lack of robustness in controlled evasion scenarios.

5. Conclusion

The comparative analysis between White Box and Black Box adversarial attacks uncovers critical insights into the strengths and weaknesses of various machine learning models in the context of adversarial spam detection. The White Box attack, due to its intimate knowledge of the target model's internal workings, posed a significantly greater challenge across all classifiers. This allowed the adversary to craft highly effective perturbations that degraded performance and revealed potential vulnerabilities in model structure and parameter sensitivity. In contrast, the Black Box attack, though still a valid and impactful adversarial strategy, had a noticeably milder effect. Most models, particularly ensemble methods like Gradient Boosting, Bagging Classifier, and Random Forest, maintained high accuracy levels and strong precision-recall balances. These models thrive on diversity and redundancy in their decision-making processes, making it harder for external perturbations to uniformly affect their output. Logistic Regression emerged as a consistent and well-rounded performer across both adversarial scenarios, confirming its robustness and adaptability in binary classification tasks. Its success stems from its straightforward optimization process and reduced risk of overfitting, allowing it to maintain reliable performance even under attack. The persistent underperformance of KNN across both adversarial setups highlights its inadequacy for real-world adversarial spam detection. The algorithm's dependence on fixed distance metrics and sensitivity to local variations make it particularly vulnerable to input perturbations and high-dimensional challenges. Furthermore, the improved performance against Black Box attacks appears to come

at a computational cost. Models such as Gradient Boosting and SVC required significantly longer training times, which may limit their practicality in real-time systems or environments with restricted computational resources. Therefore, deploying these models in operational settings necessitates a careful balance between robustness and efficiency.

In conclusion, this study underscores the importance of selecting appropriate machine learning models that can withstand different adversarial attack paradigms. While White Box attacks represent a more potent threat, the encouraging performance of ensemble methods in Black Box settings suggests that robust spam detection is achievable with current methodologies. Future work could explore adversarial training, model hardening, and hybrid defenses to enhance resilience further, while also addressing the scalability and efficiency challenges associated with more complex models.

6. Future Scope

While the current study provides a comprehensive framework for evaluating the resilience of spam detection models against black-box and white-box adversarial attacks, several avenues exist for future exploration and refinement.

One potential area of expansion involves incorporating deep learning architectures such as LSTM, GRU, and Transformer-based models (e.g., BERT) into the experimental framework. These models, known for their ability to capture long-term dependencies and contextual embeddings, could offer greater resistance or unique vulnerabilities to adversarial perturbations, especially in white-box settings where gradient information can be exploited. Investigating how these architectures respond to sophisticated attacks can further enhance the generalizability and robustness of real-world spam detection systems.

Additionally, this research can be extended to incorporate multilingual spam datasets. Spam detection is not limited to English-language content, and evaluating adversarial robustness across different languages would allow the findings to scale across global platforms. Integrating language-specific adversarial strategies may also reveal linguistic vulnerabilities that are language-dependent, which would have implications for the localization of spam detection solutions.

A promising direction lies in the domain of adversarial training, wherein models are exposed to both original and adversarial examples during training. This approach can significantly improve a model's resilience but requires further evaluation across varying levels of attack intensity and types. Techniques such as defensive distillation, input reconstruction, and randomized smoothing also merit systematic analysis in both black-box and white-box contexts to determine their practical efficacy.

Moreover, real-time deployment of robust spam classifiers on social media platforms introduces challenges related to computational efficiency and latency. Future work should explore lightweight adversarial detection mechanisms and on-device learning strategies to maintain robust performance without sacrificing user experience or system responsiveness.

Lastly, establishing an open benchmark with standardized adversarial attack libraries and diverse datasets would facilitate consistent evaluation across research studies. Such a benchmark would foster greater collaboration in the machine learning and cybersecurity communities while accelerating the development of secure NLP systems.

7. References

- [1]. Guo, Chuan, et al. "Simple black-box adversarial attacks." International conference on machine learning. PMLR, 2019.
- [2]. Narodytska, Nina, and Shiva Prasad Kasiviswanathan. "Simple Black-Box Adversarial Attacks on Deep Neural Networks." CVPR workshops. Vol. 2. No. 2. 2017.
- [3]. Wankhade, Mayur, Annavarapu Chandra Sekhara Rao, and Chaitanya Kulkarni. "A survey on sentiment analysis methods, applications, and challenges." Artificial Intelligence Review 55.7 (2022): 5731-5780.
- [4]. Zhang, Wei Emma, et al. "Adversarial attacks on deep-learning models in natural language processing: A survey." ACM Transactions on Intelligent Systems and Technology (TIST) 11.3 (2020): 1-41.
- [5]. Xu, HanXiang, et al. "Large language models for cyber security: A systematic literature review." arXiv preprint arXiv:2405.04760 (2024).
- [6]. Alzantot, Moustafa, et al. "Generating natural language adversarial examples." arXiv preprint arXiv:1804.07998 (2018).
- [7]. Ebrahimi, Javid, et al. "Hotflip: White-box adversarial examples for text classification." arXiv preprint arXiv:1712.06751 (2017).
- [8]. Zang, Yuan, et al. "Word-level textual adversarial attacking as combinatorial optimization." arXiv preprint arXiv:1910.12196 (2019).
- [9]. Vázquez-Hernández, Monserrat, et al. "White-Box Adversarial Attacks Against Sentiment-Analysis Models using an Aspect-Based Approach." Proc. ISSN 1613 (2023): 0073.
- [10]. Goyal, Shreya, et al. "A survey of adversarial defenses and robustness in nlp." ACM Computing Surveys 55.14s (2023): 1-39.